Detection of Network Intrusions and Attacks

1st Yaroslav Hayduk, 2nd María Capilla, 3rd Lukas Marović
Faculty of Information and Communication Technology, Wrocław University of Science and Technology, Wrocław, Poland
293883@student.pwr.edu.pl; maria.capzap@gmail.com; 293855@student.pwr.edu.pl

***Abstract*—Machine learning (ML) has emerged as a powerful tool for network intrusion detection, yet current approaches struggle with the fundamental trade-off between detection accuracy and computational efficiency. Static ML models fail to adapt to evolving attacks, while dynamic models that continuously process all network traffic face scalability challenges in high-throughput environments. This paper proposes a hybrid two-stage intrusion detection system that combines a lightweight static anomaly detector (Stage I) with a dynamically updated supervised classifier (Stage II). The first stage uses Isolation Forest to efficiently filter suspicious traffic with high recall, while the second stage applies adaptive ML strategies (SGD, MLP, or ensemble learning) only to pre-filtered flows. Evaluation on three datasets (NetFlow, KDD Cup 1999, and IoT-23) demonstrates that the Dynamic Ensemble achieves 95–97% balanced accuracy and near-perfect attack recall (94–100%) while reducing Stage II workload by 15–45% through intelligent pre-filtering. The proposed hybrid architecture enables practical deployment of sophisticated ML-based IDS in real-world network environments, maintaining high security while preserving computational efficiency.**

***Index Terms*—Intrusion detection, machine learning, anomaly detection, dynamic learning, network security, hybrid architecture**

I. Introduction

Modern society heavily relies on the proper functionality of communication networks, which form the backbone of information systems and support critical infrastructure, industrial control, and everyday services [1]. Ensuring their security is therefore a key requirement for maintaining continuous and reliable operation. Network infrastructures are regularly targeted and exposed to malicious activities such as jamming, signal manipulation, denial-of-service, or unauthorized access. These intrusions can cause severe disruptions in data delivery and integrity across different layers of the network stack.

Traditional signature-based or rule-based intrusion detection systems (IDS) are limited in their ability to recognize previously unseen types of attacks [2], [3]. As network technologies and threat patterns continually evolve, new types of attacks frequently appear. Consequently, these static mechanisms often fail to provide early warnings or adapt to emerging behaviors. To address these challenges, machine learning (ML) techniques have become widely used for network security monitoring, as they allow models to automatically learn from telemetry data and detect subtle deviations from normal network behavior [2], [4].

Even though ML-based intrusion detection methods can provide high accuracy for recurring attacks, they are constrained by the dynamic nature of real networks. Attack patterns and background traffic continuously change, which may lead to model drift and loss of prediction quality over time [5]. Additionally, many ML models require large, labeled datasets and extensive retraining to maintain performance. This motivates the exploration of adaptive and explainable ML approaches that can continuously update themselves, generalize across different attack types, and remain reliable in real-time operation [5]–[7].

The goal of this project is to study and evaluate ML-based methods for the detection of network intrusions and attacks. Using network telemetry datasets that include multiple types of intrusion events, the task consists of training and analyzing binary classifiers that distinguish between attack and no-attack conditions across various scenarios. The obtained results will help assess how different learning strategies maintain performance when exposed to evolving or previously unseen attack types.

II. Literature Review

Machine learning (ML) has become a fundamental component of modern network intrusion detection systems (IDS). Due to the increasing scale and sophistication of cyberattacks, traditional rule-based security mechanisms often fail to recognize novel or evolving threats [2], [3]. Recent research by Knapińska and Furdek and their collaborators explores how ML can be applied to detect attacks across different layers of communication systems, with a particular focus on adaptability, explainability, and real-time performance [5]–[7].

A. Dynamic ML Model Updating Strategies for Detection of Evolving Network Attacks

In [5], Knapińska and Furdek present two dynamic machine learning updating strategies designed to enhance the adaptability of attack detection systems. The first strategy involves incremental updates to a single classifier, while the second utilizes an ensemble method based on streaming algorithms. These approaches are validated on multiple public datasets, demonstrating that dynamic model updating significantly improves sustained detection accuracy in the face of evolving attack scenarios compared to static models. The work highlights the necessity of flexible models that can generalize across attack types and retain robust performance over time.

B. Adaptive ML for Optical Network Attack Detection

In [6], the authors experimentally evaluate the performance of adaptive ML classifiers, including Multilayer Perceptron

(MLP), Support Vector Machine (SVM), and eXtreme Gradient Boosting (XGB), to detect physical-layer attacks in optical networks. Telemetry data from a real-world testbed is used to train models in six representative attack categories. The results show that joint models trained on aggregated data can efficiently identify both known and new attacks, with MLP particularly excelling in rapid adaptation through partial fitting. Achieving up to 0.936 balanced accuracy, the research underscores the critical role of adaptive ML in maintaining network security against emerging threats.

C. Explainable and Resilient ML-Based Physical-Layer Attack Detectors

The work in [7] focuses on the explainability and resilience of ML-based attack detectors for network infrastructure. Using SHAP-based feature importance analysis, the authors enhance the interpretability of models trained on physical-layer intrusion data. They demonstrate that optimally selected feature subsets can drastically speed up detection time but warn that such streamlined models may be more susceptible to adversarial noise. The findings emphasize the importance of balancing computational efficiency and robustness against attacks, pointing to explainable AI as a crucial factor for trustworthy and effective security systems.

D. Unsupervised Network Intrusion Detection

Casas, Mazel, and Owezarski [8] discuss the limitations of IDSs that were dominant in security devices at the time. They highlight how signature-based IDS systems, which rely on known attack signatures, cannot detect new types of network attacks. They then examine anomaly-based IDS systems, which, while capable of detecting new attacks, require time-consuming training to build profiles that minimize false positives. To address these issues, the authors propose UNIDS, an unsupervised Network Intrusion Detection System designed to detect unknown attacks without labeled data, signatures, or prior training. The method combines subspace clustering, density-based clustering, and evidence accumulation clustering to identify and rank outliers in network traffic flows. When tested on the KDD'99 dataset, UNIDS successfully detected over 90% of attacks, performing similarly to a signature-based NIDS on known attacks and significantly outperforming it on unknown attacks.

III. Problem Definition

The application of machine learning (ML) to network attack detection offers significant promise, yet still confronts several critical challenges that limit the effectiveness and practicality of existing solutions. Traditional approaches that depend on static ML models, each trained for a specific known attack, suffer from both insufficient flexibility and limited scalability: it is infeasible to maintain a separate model for every possible threat in a constantly evolving attack landscape [2]. As a result, these static systems lack comprehensive coverage and quickly experience performance degradation when new or modified attack types emerge, leading to a failure in generalization and necessitating costly manual retraining from scratch each time the threat landscape changes [5].

To address the inability of static models to generalize toward unseen attacks, the field has explored unsupervised and anomaly-based learning techniques, which have demonstrated improved detection of novel threats [3], [4], [8]. Additionally, recent research has focused on incorporating data from lower layers of the network, such as the physical layer, to further enhance detection capabilities [6], [9]. However, both of these approaches are often hampered by the absence of efficient dynamic update mechanisms and either lose accuracy over time or require frequent, resource-intensive retraining, rendering them suboptimal for real-world deployment.

More recent studies introduce dynamically updating models that learn incrementally from new data, showing promising improvements in long-term performance [5]. Still, these dynamic approaches typically incur considerable computational cost because all incoming traffic is processed and used for updates. To address this open challenge, the present paper proposes a hybrid solution that combines the high efficiency and low overhead of static models for initial traffic screening with the adaptability of dynamically updating models, triggered only upon detection of suspicious behavior. This design aims to maintain effective detection of previously unseen attacks while substantially reducing the computational resources traditionally required for continuous dynamic updates.

IV. Proposed Methods

A. Overview

To address the trade-off between adaptability and computational cost in ML-based network attack detection, we propose a hybrid two-stage classification framework that combines a lightweight static anomaly detector with a dynamically updated supervised classifier. The intuition is to use an inexpensive model to continuously monitor all traffic and flag only suspicious flows, while reserving the more computationally intensive dynamic model for a smaller subset of potentially anomalous samples. In real-world scenarios most traffic is benign, so such filtering can substantially reduce the load on the adaptive component. In this way, the system aims to preserve the ability to detect evolving or previously unseen attacks, as in dynamic ML schemes [5], while reducing the overhead associated with frequent updates on the entire traffic stream.

Our approach is conceptually related to hybrid IDS architectures that chain anomaly and misuse detection components [8], [10], but it differs in several aspects. First, the first-stage detector operates directly on network telemetry features (KDD'99 and NetFlow-based datasets), focusing only on distinguishing normal from suspicious traffic rather than specific attack types. Second, the second-stage detector is not a static misuse classifier but a dynamically updated ML model, following the updating strategies proposed in [5]. This combination is intended to be both adaptable to new threats and more efficient in resource usage than purely dynamic systems.

B. Stage I: Static Anomaly Screening

The first stage of the proposed framework is a static anomaly detection model trained to characterize normal network behavior. Only benign samples are used during training, leveraging the "normal" class in KDD'99 and benign flows in the NetFlow dataset. This model is trained once offline and is not updated during operation.

In the implementation phase, we plan to experiment with classical anomaly detection techniques and density-based approaches inspired by UNIDS [8]. Features will be preprocessed using standard scaling and normalization. The decision threshold will be tuned to keep the false positive rate low, as excessive false alarms would negate its purpose as an efficient filter.

During operation, every incoming flow is first processed by the static anomaly detector. If it is classified as normal with high confidence, the flow is immediately labeled as benign and not forwarded to the second stage. Only flows flagged as suspicious are passed to the dynamic model for further analysis. This stage therefore serves as a cheap, always-on screening layer that reduces the volume of traffic that the more expensive dynamic model must handle.

C. Stage II: Dynamically Updated Binary Classifier

The second stage of the framework is a supervised binary classifier that distinguishes between attack and no-attack conditions on the subset of flows flagged as suspicious by the first stage. This model follows the dynamic updating strategies studied by Knapińska and Furdek [5]:

- **Dynamic single model:** a single classifier (e.g., MLP or SGD) updated incrementally using mini-batches of newly observed data;
- **Dynamic ensemble:** an ensemble of base learners created on consecutive batches of data, with older models gradually replaced or down-weighted.

In our framework, the key difference is that the dynamic model is trained and updated only on flows that have been pre-filtered as anomalous by Stage I. During an initial warm-up phase, the dynamic model is trained on labeled attack and benign samples from the offline training datasets, but only using instances that would be flagged as suspicious by the static detector. This simulates the real operational pipeline where the second stage never sees clearly benign traffic.

During live operation, the system continuously processes incoming flows in batches. For each batch:

1) Stage I labels each flow as normal or anomalous.
2) All anomalous flows are passed to Stage II, which outputs a binary decision (attack / no attack).
3) Once the ground-truth labels for these flows are available (e.g., in an emulated environment), Stage II is updated incrementally using only this subset, according to the chosen dynamic strategy.

The hypothesis is that by focusing the dynamic updates only on suspicious traffic, a smaller but more informative subset of data will (i) reduce training and inference time per unit of traffic, and (ii) maintain or even improve long-term detection performance compared to dynamic models trained on all flows, especially when attacks are relatively rare.

D. Baseline Methods and Evaluation Plan

To assess the effectiveness and benefits of the proposed hybrid framework, we will compare it against the following baselines:

- **Static baseline:** a conventional static supervised classifier trained once on labeled data and then applied without further updates, as commonly done in ML-based IDS studies [2], [4].
- **Dynamic-only baseline:** the dynamic single and dynamic ensemble models from [5] operating directly on the full traffic stream, without the static pre-filter.

Performance will be evaluated on standard network intrusion detection datasets (KDD'99 and NetFlow-based datasets) using metrics such as accuracy, precision, recall, F1-score, and computational cost (inference time and update time). Particular attention will be paid to the behavior of each method under evolving attack scenarios and to its ability to detect previously unseen attack types.

By quantifying both detection performance and resource consumption, we aim to determine whether the proposed hybrid approach provides a practically meaningful improvement over existing purely static or purely dynamic ML-based intrusion detection systems.

V. Solution Implementation

A. Data Preprocessing

Before training, each dataset underwent a preprocessing pipeline to ensure data quality and consistency. The preprocessing steps included:

Feature Selection: We removed features that would cause overfitting or provide no discriminative power:

- **NetFlow:** dropped identifiers (FLOW_ID, ID), metadata (ALERT, ANALYSIS_TIMESTAMP), address/port fields (IPV4_SRC_ADDR, IPV4_DST_ADDR, L4_SRC_PORT, L4_DST_PORT), and zero-variance fields (MIN_IP_PKT_LEN, MAX_IP_PKT_LEN).
- **KDD:** removed constant-value fields (land, num_outbound_cmds, is_host_login) and near-zero variance fields with zero importance (su_attempted, wrong_fragment, num_failed_logins, rerror_rate).

Encoding: Categorical features were encoded using Label Encoding. For KDD, this included protocol_type, service, and flag columns. The encoders were fitted on training data and applied to test data to prevent data leakage.

Scaling: All numerical features were normalized using RobustScaler, which is less sensitive to outliers than standard scaling. This is important for network traffic data where extreme values (e.g., large byte counts during attacks) are common.

Label Conversion: Attack labels were converted to binary format (0 = normal, 1 = attack). For KDD, this involved mapping the 23 different attack types to a single attack class.

B. Stage I: The Filter

For the initial development of our solution, we focused on the first stage: training an unsupervised Isolation Forest model to detect anomalies for further analysis. We selected Isolation Forest because this algorithm excels at outlier detection. We trained it using 80% of the RoEduNet-SIMARGL2021 dataset [11], after removing all anomalous traffic (to avoid biasing the model toward known anomalies) and irrelevant features that introduced noise—this feature engineering was the most challenging aspect of Stage I development. The model was configured with 100 estimators and a contamination parameter tuned to 0.5 to favor high recall on anomalous traffic. We then tested it on a dedicated test dataset with the anomaly distribution shown in Figure 1.

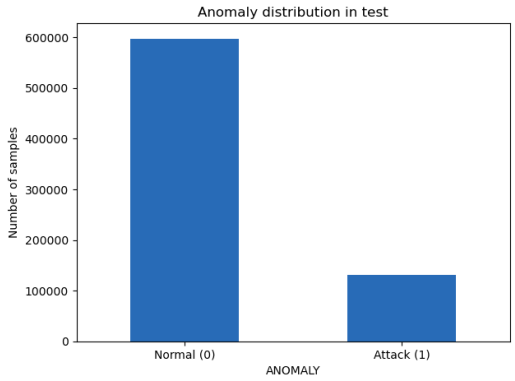

Fig. 1. Anomaly distribution in test dataset.

For testing our model, we tried multiple values of contamination, and we obtained the results displayed in Figure 2.

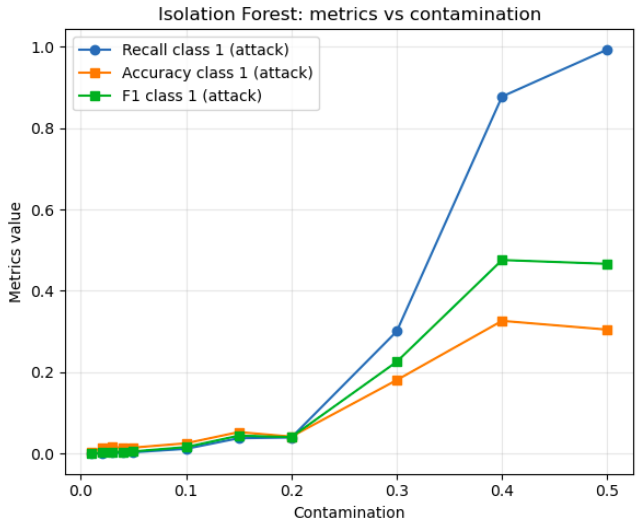

Fig. 2. Isolation Forest: metrics vs contamination (class 1).

In this context, contamination = 0.45 is the most appropriate value because it maximizes the model's ability to detect attacks, which is the primary goal in an intrusion detection system. At 0.45, the recall for the attack class (class 1) reaches its highest value, meaning the model misses far fewer attacks than with lower contamination values, and the F1-score for attacks is also maximized, indicating the best overall balance between precision and recall for that class. Although precision for attacks is slightly lower than at 0.4, the significant gain in recall and F1-score is more important, since in this domain failing to detect attacks is more critical than generating some additional false positives.

Although the chosen configuration generates a relatively high number of false positives, this behavior is acceptable—and even desirable—in our architecture. The anomalies flagged by the Isolation Forest in Stage 1 (including false positives) will be used as additional labelled data for the Stage-2 model, which is designed to learn dynamically and refine the decision boundaries between normal and attack traffic. Therefore, Stage 1 acts as a high-recall, coarse filter that feeds a more powerful supervised classifier, rather than as a final detector on its own.

Also, this model still filters out around 50% of benign traffic, which significantly reduces the computational cost of the second-stage model, since that stage only needs to process the remaining, more suspicious flows shown in Figure 3.

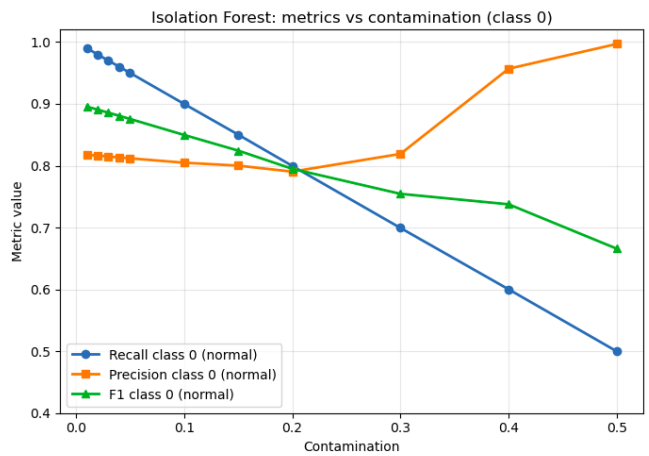

Fig. 3. Isolation Forest: metrics vs contamination (class 0).

C. Stage II: The Expert

The second stage of the solution, which should discern false positives from real attacks, supports multiple strategies, allowing comparison between different levels of complexity and adaptability. The strategies that we have chosen for experimentation are two dynamic single models and a dynamic ensemble model:

1) SGD (Stochastic Gradient Descent) – a linear dynamic single classifier for fast, linear updates.
2) MLP (Multi-layer Perceptron) – a neural network-based dynamic single classifier for capturing non-linear patterns.
3) Dynamic ensemble – a strategy using an SEA-inspired pool of classifiers that trains a new model for each batch

and prunes the worst performer, offering high stability. Prediction is done using a majority vote.

The pipeline for a chosen strategy consists of two phases: a warm-up phase and an online testing phase. The warm-up phase consists of initializing our dynamic models using training data that the Isolation Forest from the first stage flagged as anomalous. The models need to be trained with some initial data, or else they will not be able to comprehend the inputs during the second phase. For the dynamic ensemble strategy this means creating an initial pool of classifiers by splitting the training data up into chunks for each new classifier to be trained on.

In the online testing (simulation) phase, the test data is split into batches which are processed one by one. The batch data is first processed by the first stage and is then sent to the dynamic model for prediction. The dynamic model is then updated using the test data, and both the predicted labels and the true labels are stored for further evaluation. The dynamic single models are updated using partial fitting, while the dynamic ensemble is updated by replacing the worst performing model in the pool with a new one trained on the batch data.

The three dynamic models show different behaviors depending on the batch size used, as shown in Figures 4, 5, and 6.

SGD Classifier: This model is sensitive to batch size. It works best with batch size 25,000, reaching 87.38% balanced accuracy and 80.55% attack recall. Smaller batches give worse results, and batch size 20,000 shows a drop in recall to 60.44%. This happens because SGD needs enough data per batch to update the weights properly [12].

MLP Classifier: The MLP is more stable than SGD across different batch sizes. The best results are at batch size 20,000 with 90.85% balanced accuracy and 87.24% attack recall. Even with other batch sizes, recall stays above 77%, showing that neural networks handle incremental learning better [6], [13].

Dynamic Ensemble: The ensemble gives the best and most consistent results. Balanced accuracy stays above 96% and attack recall above 97.6% for all batch sizes tested. This stability comes from using multiple classifiers and majority voting [14], [15].

Fig. 4. SGD: metrics vs batch size (class anomaly).

VI. Results

Table I shows the best configuration for each model on the RoEduNet-SIMARGL2021 dataset.

A. Analysis

The Dynamic Ensemble has the best detection performance with 97.66% attack recall and 96.09% balanced accuracy.

Fig. 5. MLP: metrics vs batch size (class anomaly).

Fig. 6. Ensemble: metrics vs batch size (class anomaly).

TABLE I
Comparison of optimal model performance metrics.

Metric	SGD	MLP	Ensemble
Optimal Batch Size	25,000	20,000	10,000
Balanced Accuracy (%)	87.38	90.85	96.09
Precision (Anomaly)	0.7543	0.7768	0.7974
Recall (Anomaly) (%)	80.55	87.24	97.66
F1-Score (Anomaly)	0.7791	0.8218	0.8780
Total Processing Time (s)	11.61	17.87	151.01

MLP comes second with 87.24% recall, and SGD is last with 80.55%. Attack recall is the most important metric for an IDS because missing an attack is worse than a false alarm [16], [17].

However, better detection comes at a higher computational cost. SGD processes the data in 11.61 seconds, MLP in 17.87 seconds, and the ensemble takes 151.01 seconds. The ensemble is about 13 times slower than SGD.

The Stage I filter helps reduce this problem by filtering out 45.25% of benign traffic before it reaches Stage II. This makes the ensemble feasible for real-time use since it only processes suspicious flows [9], [18].

Each model fits different use cases: SGD for high-speed, low-security scenarios; MLP for balanced speed and accuracy; and the ensemble for high-security environments where missing attacks is not acceptable [2], [4].

B. Cross-Dataset Generalization

To validate the generalization capability of the proposed hybrid framework, we extended our evaluation to two additional datasets: the KDD Cup 1999 dataset [8] and the IoT-23 dataset. Table II presents the performance of each Stage II model across all three datasets.

The results demonstrate that the Dynamic Ensemble consistently achieves the highest balanced accuracy across all datasets, ranging from 95.31% on IoT-23 to 97.66% on NetFlow. The ensemble also maintains attack recall above 94% on all datasets, with near-perfect recall (99.98%) on the IoT-23 dataset. This confirms that the SEA-inspired pruning strategy and majority voting mechanism provide robust generalization across different network environments and attack patterns.

TABLE II
CROSS-DATASET COMPARISON OF STAGE II MODEL PERFORMANCE.

Dataset	Model	Bal. Acc.	F1 Macro	Recall	Time (s)
KDD	SGD	0.9013	0.8741	0.9246	**16.75**
	MLP	0.9542	0.9231	**0.9447**	27.79
	Ensemble	**0.9613**	**0.9262**	0.9429	372.77
IoT-23	SGD	0.9035	0.9039	0.9174	**5.05**
	MLP	0.8938	0.8921	0.8725	6.62
	Ensemble	**0.9531**	**0.9558**	**0.9998**	55.46
NetFlow	SGD	0.8308	0.8436	0.7055	**11.07**
	MLP	0.9310	0.9188	0.9004	14.93
	Ensemble	**0.9766**	**0.9596**	**0.9789**	130.14

The Stage I Isolation Forest filter demonstrates consistent behavior across datasets. On KDD, 85.04% of samples are passed to Stage II; on IoT-23, 74.65%; and on NetFlow, 54.64%. This filtering reduces the computational burden on the dynamic classifiers while maintaining high attack recall at Stage I (above 97% across all datasets).

Notably, the computational overhead of the ensemble remains manageable due to the Stage I pre-filtering. Without the filter, the ensemble would need to process all incoming traffic, significantly increasing processing time. The hybrid architecture thus enables the use of more sophisticated but slower classifiers in practical deployment scenarios.

VII. CONCLUSION

This paper presented a hybrid two-stage intrusion detection system that combines a static Isolation Forest anomaly detector with dynamically updated supervised classifiers. The proposed architecture addresses the trade-off between detection accuracy and computational efficiency by using Stage I as a high-recall filter that reduces the volume of traffic processed by the more expensive Stage II models.

Experimental evaluation across three datasets (NetFlow, KDD, and IoT-23) demonstrated that the Dynamic Ensemble consistently achieves the best detection performance, with balanced accuracy ranging from 95.31% to 97.66% and attack recall above 94%. The MLP classifier provides a good balance between accuracy and speed, while SGD offers the fastest processing for less critical applications.

The Stage I filter successfully reduces Stage II workload by 15–45% while maintaining attack recall above 97%, validating our hypothesis that a lightweight anomaly detector can effectively pre-screen traffic without sacrificing detection capability. This makes the computationally intensive ensemble approach feasible for practical deployment.

For practical deployment, we recommend a context-aware model selection strategy: the Dynamic Ensemble is best suited for high-security environments or specialized network segments with moderate traffic volumes (e.g., critical infrastructure, IoT networks, or specific enterprise subnets) where near-perfect attack detection justifies the computational cost. In contrast, the MLP classifier offers an optimal balance for large-scale, high-throughput networks where real-time processing of massive traffic flows is essential, providing strong detection performance (87–94% attack recall) with significantly lower computational overhead. The SGD classifier remains viable for resource-constrained deployments where speed is paramount and slightly reduced detection rates are acceptable.

REFERENCES

[1] C. Alcaraz and S. Zeadally, "Critical infrastructure protection: Requirements and challenges for the 21st century," *International Journal of Critical Infrastructure Protection*, vol. 8, no. 4, pp. 53–66, 2015.

[2] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 20, pp. 1–22, 2019.

[3] R. Samrin and D. Vasumathi, "Review on anomaly based network intrusion detection system," in *2017 International Conference on Electrical, Electronics, Communication, Computer, and Optimization Techniques (ICEECCOT)*, 2017, pp. 141–147.

[4] T. Zoppi, G. Fenza, D. Furno, V. Loia, F. Orciuoli, and S. Senatore, "Which algorithm can detect unknown attacks? comparison of supervised, unsupervised and meta-learning algorithms for intrusion detection," *Computers & Security*, vol. 127, p. 103107, 2023.

[5] A. Knapińska and M. Furdek, "Dynamic ml model updating strategies for detection of evolving network attacks," in *2025 15th International Workshop on Resilient Networks Design and Modeling (RNDM)*, 2025, pp. 1–7.

[6] A. Knapińska and M. Furdek, "Experimental analysis of adaptive ml classifiers for dynamic detection of emerging physical-layer attacks," *Conference Proceedings or Journal Name*, pp. 1–4, 2025.

[7] A. Knapińska and M. Furdek, "Explainable and resilient ml-based physical-layer attack detectors," 2025.

[8] P. Casas, J. Mazel, and P. Owezarski, "Unsupervised network intrusion detection systems: Detecting the unknown without knowledge," *Computer Communications*, vol. 35, no. 7, pp. 772–783, 2012.

[9] C. Natalino, D. Siracusa *et al.*, "Root cause analysis for autonomous optical network security management," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2702–2713, 2022.

[10] S. T. Powers and J. He, "A hybrid artificial immune system and self organising map for network intrusion detection," *Information Sciences*, vol. 178, no. 15, pp. 3024–3042, 2008.

[11] M.-E. Mihailescu, D. Mihai, M. Carabas, M. Komisarek, M. Pawlicki, W. Holubowicz, and R. Kozik, "The proposition and evaluation of the roedunet-simargl2021 network intrusion detection dataset," *Sensors*, vol. 21, no. 13, p. 4319, 2021.

[12] T. Zhang, "Solving large scale linear prediction problems using stochastic gradient descent algorithms," in *International Conference on Machine Learning (ICML)*, 2004.

[13] G. E. Hinton, "Connectionist learning procedures," in *Machine Learning*. Elsevier, 1990, pp. 555–610.

[14] W. N. Street and Y. Kim, "A streaming ensemble algorithm (sea) for large-scale classification," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2001, pp. 377–382.

[15] G. Ditzler, M. Roveri, C. Alippi, and R. Polikar, "Learning in non-stationary environments: A survey," *IEEE Computational Intelligence Magazine*, vol. 10, no. 4, pp. 12–25, 2015.

[16] S. J. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. K. Chan, "Cost-based modeling and evaluation for data mining with application to fraud and intrusion detection: Results from the jam project," 1999.

[17] G. M. Weiss, "Mining with rarity: A unifying framework," *ACM SigKDD Explorations Newsletter*, vol. 6, no. 1, pp. 7–19, 2004.

[18] P. Mulinka and P. Casas, "Stream-based machine learning for network security and anomaly detection," in *Big-DAMA Workshop*, 2018.